

CSCE 5200: INFORMATION RETRIVAL AND WEB SEARCH

PROJECT REPORT: SIMPLE IR ENGINE

Author

Maddineni Naga Sampath
University of North Texas, Denton
NagaSampathMaddineni@my.unt.edu

1. Introduction:

An IR Engine consists of three essential elements which include parser, indexer and Retriever. Our project features three distinct stages that follow the design of an IR engine. The text parsing starts the first phase while forward indexing and inverted indexing techniques operate in phase two. In the third phase we retrieve the documents based on the query. During Phase1 A large document collection found in ft911/ directory gets processed to extract text data which enables the creation of two index types called Forward Index and Inverted Index in phase2. The project aims to create an efficient document processing system which supports quick term-based document retrieval. This report documents every step of IR engine development starting from design to implementation and performance testing for both the index size and search functionality and operation time.

2. Design and Implementation:

The project consists of three main Phases:

2.1. Phase 1: Text Parser:

This phase tokenizes and stems text documents to create distinct vocabulary along with dictionaries for every document. The ft911/ directory provides our documents alongside their html tags and other elements inside this folder using RegEx. We need to extract the documents with their associated ids and texts before performing Tokenization. The text processing includes converting all text to lowercase form while removing digits and dividing the text into tokens using any alphanumeric sequence. The following step involves stop word elimination because stop words refer to standard words (stop words) that we acquire from stopwords.txt file to concentrate on essential meaning in texts. We proceed to stemming where the Porter stemmer reduces words to their root bases. We code the document processor to extract documents by utilizing RegEx, then do the above steps, then tokenize and move onto the producing the word or term dictionary and doc dictionary.

Doc Dictionary: It notes down the doc's name as key and assigns a unique value, in our case it's the same number in doc name.

Term Dictionary: It notes down all the unique terms in all docs in order sorted as keys and then assigns a unique value to them in the order. Here in this phase, we also output the term dict and doc dict to the parser_output.txt file

2.2. Phase 2: Indexer:

Here in this phase, we design to build the forward index and inverted index utilizing phase1 and build upon it. Also, with that we calculated the time taken for building both indexes and the file sizes of the indexes. We extend the phase and create a new python program which contains the functions for building both indexes by passing the processed i.e., tokenized documents from phase 1 and term dictionary.

Forward Index: The frequency information for every term in a document gets stored within this index. The index contains a system that establishes a relationship between document IDs and term frequency counts. Any word in a processed document is counted by the indexer which saves this information in a dictionary that pairs document IDs with word frequencies.

Inverted Index: The index maintains two components: it lists each document that contains terms and includes the term frequency within each document. During its operation the indexer processes all documents to generate

an inverted index that associates each word ID with a list of documents which contain that word and their respective word frequencies.

Here we have also given functionality to search given a query word, it looks up the word in the inverted index and returns a list of documents that contain the term. The indexing process requires a stepwise execution of total time calculations for parsing and indexing followed by forward and inverted index creation and index file disk storage and index file size printing.

3. Performance Evaluation:

The system requires Parsing Time to evaluate and analyze all documents as well as extracting relevant information from them. Building the forward index requires time to process tokens and count their frequencies during Forward Index Construction Time. The time required for building an inverted index that matches terms to document IDs and frequencies is referred to as Inverted Index Construction Time.

In total, the entire indexing process (from parsing to saving indexes) was completed in 19.49 seconds, with a breakdown as follows (the values for each run time)

Parsing: ~18.16 seconds

Forward Indexing: ~0.36 seconds

Inverted Indexing: ~0.51 seconds

Saving indexes in Required format: ~0.47 seconds

Index Size: The sizes of the generated indexes were calculated to be constant.

Forward Index: The size of the forward index file was ~5731 KB.

Inverted Index: The size of the inverted index file was ~5363 KB.

These sizes indicate that the indexing process is relatively efficient given the size of the input data. We can also the time taken for inverted indexing is more than that of forward indexing. Also, the time taken for parsing is high because it also contains downloading punkt, extracting docs, tokenizing, stop word removal and finally creating term and doc dict. Thus, it has a high time than indexing. But in these short times we can say the process is efficient.

4. Results:

The IR engine consists of basic search capabilities. Users can submit terms to the engine which fetches documents that contain those terms. The search process involves:

- i. The word ID for the query term can be accessed through word_dict.
- ii. A lookup operation on the inverted index uses the word ID to identify all documents which include the term.
- iii. The system prints the documents together with their term frequencies after matching the specified term.

Example: I have tried this on the term “bat”, got the inverted index entries through the lookup in inverted index, The result as follows Documents containing 'bat' (word ID 2300): {12: 1, 524: 1, 875: 1, 1425: 1, 1476: 1, 2002: 1, 2480: 1, 2653: 1, 2784: 1, 2804: 4, 3004: 1, 3692: 1, 3912: 1, 4089: 1, 4212: 1, 4657: 1, 4741: 1, 5102: 1}.

```
PS C:\Users\SampathNagaMaddineni\Downloads\proj1> python indexer.py
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\SampathNagaMaddineni\AppData\Roaming\nltk_dat
[nltk_data] a...
[nltk_data] Package punkt is already up-to-date!
Parsing completed in 18.16 seconds.
Forward Indexing completed in 0.36 seconds.
Inverted Indexing completed in 0.51 seconds.
Total Indexing completed in 0.87 seconds.
Saving indexes completed in 0.47 seconds.
Size of forward_index.txt: 5731.22 KB
Size of inverted_index.txt: 5363.87 KB
Total time elapsed: 19.49 seconds.
Indexes built and saved in required format.
Enter the term you want to search:
bat
Documents containing 'bat' (word ID 2300): {12: 1, 524: 1, 875: 1, 1425: 1, 1476: 1, 2002: 1,
2480: 1, 2653: 1, 2784: 1, 2804: 4, 3004: 1, 3692: 1, 3912: 1, 4089: 1, 4212: 1, 4657: 1,
4741: 1, 5102: 1}
PS C:\Users\SampathNagaMaddineni\Downloads\proj1>
```